

软件工程基础

第 4 章—软件项目计划

Software Project Planning

学习笔记 / Study Notes

中南大学计算机学院课程整理

2026 年 6 月 1 日

核心知识框架：风险分析（标识 → 估计 → 评价 → 管理） | PERT 关键路径/Gantt 图进度安排 | Brooks 定律与五种项目组织形式

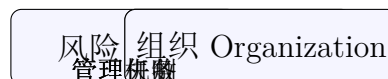
目录

1	软件项目计划概述 Overview of Software Project Planning	2
2	风险分析 Risk Analysis	3
2.1	风险标识 Risk Identification	3
2.1.1	风险分类 (Risk Categories)	3
2.1.2	Bohem 风险项目检查表 (Risk Item Checklist)	3
2.1.3	头脑风暴法 (Brainstorming)	4
2.2	风险估计 Risk Estimation	4
2.3	风险评价 Risk Evaluation	5
2.3.1	风险三元组 (Risk Triplet)	5
2.3.2	风险参考水准曲线 (Reference Level Curve)	5
2.3.3	建立风险表 (Risk Table)	5
2.4	风险管理与监控 Risk Management & Monitoring	6
2.4.1	风险管理策略	6
2.4.2	风险监控 (Risk Monitoring)	6

3	进度安排 Schedule Planning	7
3.1	基本原则 Basic Principles	7
3.2	工作量分配: 40-20-40 规则 40-20-40 Rule	7
3.3	PERT 技术 PERT: Program Evaluation & Review Technique	8
3.3.1	PERT 图基本元素	8
3.3.2	EET (Earliest Event Time) 最早时刻	8
3.3.3	LET (Latest Event Time) 最迟时刻	8
3.3.4	关键路径 (Critical Path)	9
3.3.5	机动时间 (Slack Time)	9
3.4	Gantt 图方法 Gantt Chart	9
4	项目组织 Project Organization	11
4.1	人员组织规律 Personnel Organization Principles	11
4.1.1	Rayleigh-Norden 曲线	11
4.1.2	人员-时间权衡定律 (Personnel-Time Tradeoff)	11
4.1.3	Brooks 定律 (Brooks' Law)	12
4.2	五种项目组织形式 Five Organizational Structures	12
4.2.1	对比总表	14
4.2.2	五种形式图示	14
5	软件项目开发计划文档 Project Plan Document	15
6	小结 Summary	15

1 软件项目计划概述 Overview of Software Project Planning

- **目标 (Goal):** 为管理者提供一个框架, 使其能够对**资源**、**成本**、**风险**及**进度**进行合理的估算分析和调度, 为软件工程过程提供管理依据。 [p.3]
- **制定者:** 软件项目管理员、系统分析员与用户共同制订, 在需求分析阶段弄清详细情况后才能正式定稿。 [p.3]
- **四大作用:** [p.4]
 - (1) 减少风险损失 (风险/Risk)
 - (2) 提供明确目标 (任务/Task)
 - (3) 高效利用资源, 提高效益 (进度/Schedule)
 - (4) 促进成员配合, 体现综合效应 (组织/Organization)
- **计划三核心内容:** [p.5]



2 风险分析 Risk Analysis

“如果你不主动攻击风险，风险就会主动攻击你。”——T. Gilb [p.6]

风险管理策略：在技术工作开始之前，先标识潜在风险，评估概率和影响，按重要性排序，再制订管理计划。 [p.6]

2.1 风险标识 Risk Identification

2.1.1 风险分类 (Risk Categories)

[p.7]

表 1: 风险的两个分类维度

维度	类别	示例
2* 宏观分类 (By Scope)	项目风险 (Project Risk): 预算、进度、人员、资源	资金不足
	技术风险 (Technical Risk): 设计、实现、接口、维护	规格说明存在二义性
	商业风险 (Business Risk): 市场、策略、管理、预算	开发的产品过时
2* 可预测性 (Charette)	已知风险 (Known Risk): 通过评估可以发现	用户计算机知识欠缺
	可预测风险 (Predictable Risk): 从过去经验推断	用户改变需求
	不可预测风险 (Unpredictable Risk): 极难事先识别	自然灾害

2.1.2 Bohem 风险项目检查表 (Risk Item Checklist)

[p.8]

- **产品规模 (Product Size):** 与待开发或修改软件总体规模相关的风险。
- **商业影响 (Business Impact):** 与管理或市场所加约束相关的风险。
- **客户特性 (Customer Characteristics):** 与客户素质及开发者与客户定期通信能力相关的风险。
- **过程定义 (Process Definition):** 与软件过程被定义程度及遵守程度相关的风险。

- **开发环境 (Development Environment)**: 与建造产品工具的可用性及质量相关的风险。
- **建造技术 (Construction Technology)**: 与软件复杂性及系统所包含技术”新奇性”相关的风险。
- **人员数量及经验 (Personnel Number & Experience)**: 与参与工作软件技术人员的总体水平及项目经验相关的风险。

三大风险对照检查表:

[p.9-11]

1. **商业风险表**: 市场时机、产品过时、竞争压力等。
2. **技术风险表**: 技术新颖性、算法复杂度、接口复杂性等。
3. **管理风险表**: 资源分配、沟通效率、团队稳定性等。

2.1.3 头脑风暴法 (Brainstorming)

[p.12]

- 召集项目主要参与人员代表。
- 利用各自对项目不同部分的认识, 识别可能出现的问题。
- 轮流发言, 无条件接纳任何意见, 不加评论。
- 专人在白板上记录每一条意见。
- 成员可不发表意见而跳过 (Pass)。
- 过程持续直到所有人无新意见或限定时间到。

其他方法: Delphi 法 (Delphi Method)、事故树分析法 (Fault Tree Analysis)、因果图法 (Cause-Effect Diagram)、情景分析法 (Scenario Analysis) 等。

[p.12]

2.2 风险估计 Risk Estimation

- **风险概率 (Probability)**: 估计风险发生的可能性, 用布尔值、定量或定性的方式表示。

[p.13]
- **风险影响 (Impact)**: 估计问题出现后带来的损失——**灾难的** (Catastrophic)、**严重的** (Severe)、**轻微的** (Marginal)、**可忽略的** (Negligible)。

[p.13]

2.3 风险评价 Risk Evaluation

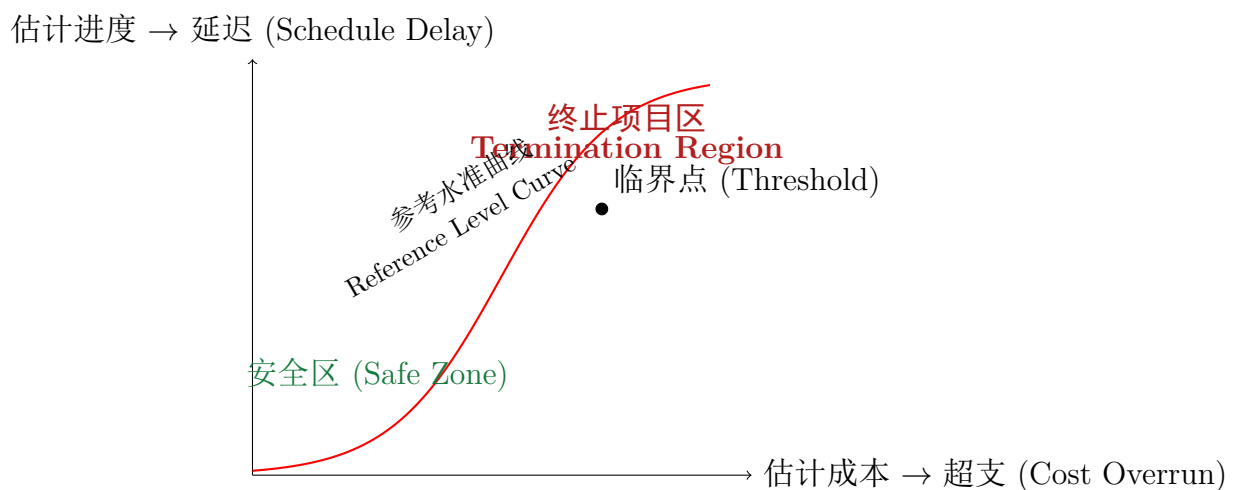
2.3.1 风险三元组 (Risk Triplet)

[p.15]

- 三元组定义: $[r_i, p_i, e_i]$
 - r_i —— 风险 (Risk)
 - p_i —— 风险出现的概率 (Probability)
 - e_i —— 风险产生的影响 (Effect/Impact)
- 定义项目各种风险**参考水准** (Reference Level), 如成本、进度等。
- 预测一组临界点以定义**项目终止区** (Termination Region), 由曲线或易变动区域界定。
- 预测怎样的风险组合会影响参考水准。

2.3.2 风险参考水准曲线 (Reference Level Curve)

[p.19]



曲线含义: 当项目成本超支和进度延迟的组合超过参考水准曲线时, 应考虑终止项目; 曲线下方为可接受区域。

2.3.3 建立风险表 (Risk Table)

[p.17-18]

1. **估算风险概率值**: 由开发团队各成员各自估算, 然后投票, 直到估算值接近为止。
2. **确定影响类别**: 针对性能 (Performance)、支持 (Support)、成本 (Cost) 及进度 (Schedule) 四个影响因素, 分别确定影响类别。
3. **求平均**: 将 4 个影响因素的影响类别求平均, 得到整体的影响类别。

2.4 风险管理与监控 Risk Management & Monitoring

2.4.1 风险管理策略

[p.20]

- **利用技术手段**: 原型化 (Prototyping)、软件自动化 (Software Automation)、可靠性工程学 (Reliability Engineering) 等。
- **成本收益原则**: 仅当风险管理所需成本 < 风险带来的影响时才可以实施。
- **优先级**: 高影响且中-高概率的风险, 以及低影响但高概率的风险, 应**首先列入管理**。
- **Pareto 80-20 规则**: 80% 的软件风险能够由仅仅 20% 的已标出风险来说明。 [p.20]

2.4.2 风险监控 (Risk Monitoring)

[p.21]

- **跟踪 (Tracking)**: 跟踪事件和主要风险因素, 判断预测的风险是否确实发生。
- **控制 (Control)**: 确保针对某个风险制定的风险管理措施正在实施。
- **收集 (Collection)**: 收集可用于将来风险分析的信息。

3 进度安排 Schedule Planning

3.1 基本原则 Basic Principles

[p.23]

1. **任务分解** (Task Decomposition): 将软件工程项目任务分解为易管理的子任务 (作业/Activity)。
2. **作业依存** (Activity Dependency): 确保作业间的依存关系 (顺序/Sequential 和并发/Concurrent)。
3. **时间分配** (Time Allocation): 为每个作业指定开始和终止时间。
4. **资源约束** (Resource Constraints): 考虑资源约束, 如人员数量、工具。
5. **定义责任** (Responsibility Definition): 指定某特定小组或个人负责某个作业。
6. **定义结果** (Output Definition): 对每个作业定义相应的结果 (产品或产品的一部分)。
7. **定义里程碑** (Milestone Definition): 每个作业或作业系列应与项目的里程碑相联系。

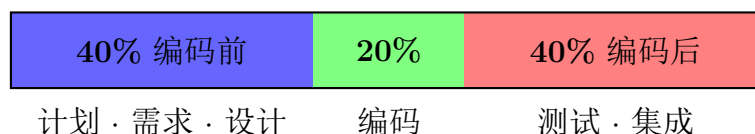
3.2 工作量分配: 40-20-40 规则 40-20-40 Rule

[p.24]

40-20-40 工作量分配规则

在整个软件开发过程中:

- **编码前工作** (Pre-Coding): **40%** —— 计划 (Planning)、需求分析 (Requirements Analysis)、设计 (Design)
- **编码工作** (Coding): **20%** —— 编码实现 (Coding/Implementation)
- **编码后工作** (Post-Coding): **40%** —— 测试 (Testing)、集成 (Integration)、维护准备 (Maintenance Preparation)



3.3 PERT 技术 PERT: Program Evaluation & Review Technique

[p.27-32]

PERT（计划评审技术/工程网络技术），20 世纪 50 年代后期由美国海军和洛克希德公司首次提出，成功应用于北极星导弹研发。

[p.27]

3.3.1 PERT 图基本元素

[p.28]

3.3.4 关键路径 (Critical Path)

[p.31]

- **关键事件**：EET = LET 的事件。
- **关键路径**：由关键事件串联而成的路径（PERT 图中最长的路径）。
- 关键路径上的任何延迟 = 整个项目的延迟。
- PERT 图中以**粗线箭头**标示关键路径。

Gantt 图特点:

- **优点:** 直观、易懂, 适合向管理层汇报。
- **缺点:** 不直接显示任务间的依赖关系, 难以反映关键路径。
- **最佳实践:** 与 PERT 图配合使用——PERT 图分析关键路径, Gantt 图跟踪实际进度。

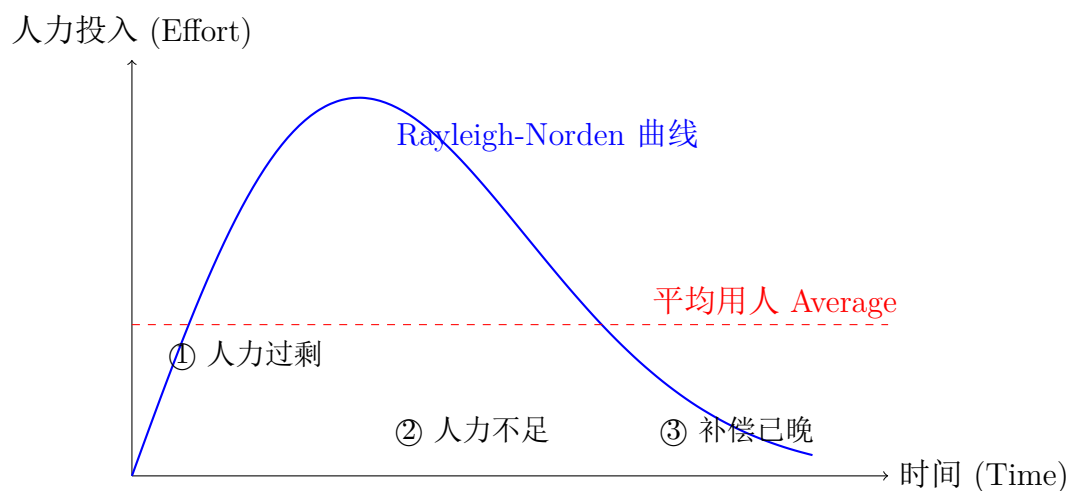
4 项目组织 Project Organization

项目组织中人员管理不能像其他事务那样简单划分、机械对待 (DeMarco)。 [p.37]

4.1 人员组织规律 Personnel Organization Principles

4.1.1 Rayleigh-Norden 曲线

[p.38]



核心结论：如果按平均用人水平分配人力资源：

- ① 起始阶段人力过剩（浪费）。
- ② 开发后期和维护前期人力不足（瓶颈）。
- ③ 维护期补偿人力为时已晚。

各类人员参与程度：

[p.39]

- 初级技术人员：编码时参与最多，其他阶段较少。
- 高级技术人员：开始阶段和结束阶段参与较多，中间阶段较少。
- 管理人员：项目开始阶段参与最多。

4.1.2 人员-时间权衡定律 (Personnel-Time Tradeoff)

[p.40]

$$E \propto \frac{1}{t^4}$$

其中 E = 软件开发工作量 (Effort), t = 软件开发时间 (Time)。

- 软件开发工作量与开发时间的 4 次方成反比。
- 适当延长开发时间 $\Rightarrow$ 减少工作量。
- 缩短开发时间 $\Rightarrow$ 工作量大幅度增加。

4.1.3 Brooks 定律 (Brooks' Law)

[p.41]

Brooks 定律

向一个已经延期的软件项目追加开发人员，可能会使它完成得更晚。

Adding manpower to a late software project makes it later.

原因：当开发人员以算术级数 (Arithmetic Progression) 增长时，人员之间的通信将以几何级数 (Geometric Progression) 增长。

$$\text{通信路径数} = \frac{n(n-1)}{2} \quad (n = \text{人员数量})$$

推论：对于软件项目，**开发时间宁可长一些，开发人员可少而精一些。** [p.41]

4.2 五种项目组织形式 Five Organizational Structures

[p.42-47]

组织设计三原则：

[p.42]

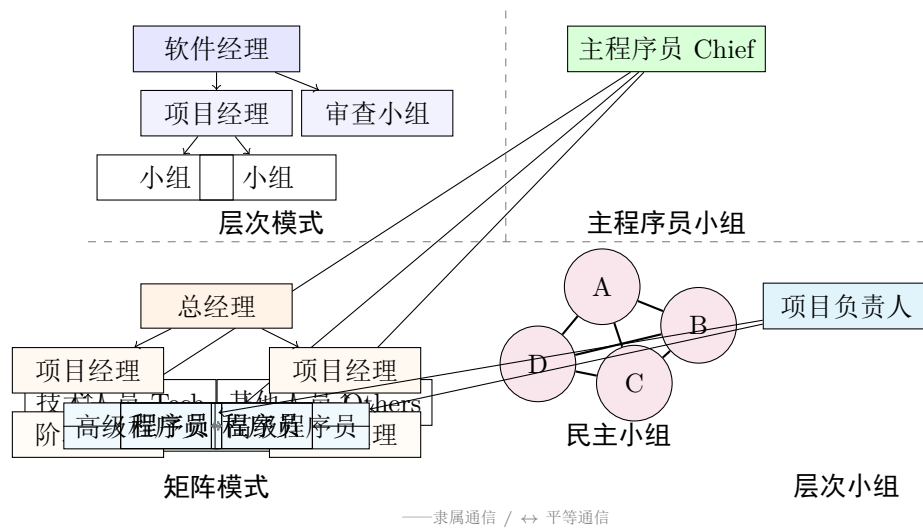
1. 责任到人：尽早落实责任，便于管理。
2. 合理分工：减少不必要的通信，提高工作效率。
3. 责权均衡：责任与权力的平衡，有助于任务完成。

表 2: 五种项目组织形式对比

组织形式	结构特点	通信方式	适用场景	优缺点
层次模式 (Hierarchical) [p.43]	软件经理 → 项目经理 → 小组（审查小组并列）	自上而下，层级通信	大型组织，多项目并行	结构清晰，但层层传递可能低效
矩阵模式 (Matrix) [p.44]	纵横交叉：项目经理（问题域）+ 阶段经理（技术域）	双重汇报	多项目共享专业资源	资源利用率高，但可能多头领导
主程序员小组 (Chief Programmer) [p.45]	主程序员 + 后备程序员 + 技术人员/其他	主程序员中心化	中小型项目，需要强技术领导	效率高，但过度依赖主程序员（H. Mills/Baker 提出，IBM 1971 首次应用）
民主小组 (Democratic) [p.46]	无中心领导，平等协作	全连通，平等通信	创新性项目，团队高素质	士气高（“无我程序设计”/Egoless Programming, Weinberg 1971），但决策慢
层次小组 (Hierarchical Team) [p.47]	项目负责人 → 高级程序员 → 程序员	层级间隶属通信，组内平等通信	大型项目，层次结构课题	兼顾效率和灵活性，适合大型项目

4.2.1 对比总表

4.2.2 五种形式图示



5 软件项目开发计划文档 Project Plan Document

[p.48]

- **目的：**用文件形式记录各项目工作的**负责人员、开发进度、经费预算、所需软/硬件条件**等，以便据此开展和检查项目开发工作。
- **时机：****只有在需求分析之后才能正式定稿。**
- **特性：**计划文档可能随项目进展而**不断调整**。

6 小结 Summary

[p.49]

- **风险分析：**做到”知己知彼”（彼即风险），从而”百战不殆”，使开发者能战胜风险带来的损失，让项目成功。
- **进度安排：**落空不仅造成项目开发成本提高（有形的经济损失），还会使客户不满、不信任增加（无形的经济损失）。
- **项目组织：**在软件开发过程中，**人是最活跃的部分**。

核心公式与定律速查表

资料来源：中南大学计算机学院任胜兵《软件工程基础》第 4 章课件（49 页）

整理时间：2026 年 6 月 1 日

名称	公式/表述	含义
风险三元组	$[r_i, p_i, e_i]$	风险 = (风险本身, 概率, 影响)
EET 计算	$EET(i) = \max\{EET(k) + d(k, i)\}$	最早时刻 = 前驱最早时刻 + 持续时间的最大值
LET 计算	$LET(i) = \min\{LET(j) - d(i, j)\}$	最迟时刻 = 后继最迟时刻 - 持续时间的最小值
机动时间	$Slack = LET - EET - d$	作业可延迟时间
关键路径	EET = LET 的事件串联	项目最长路径, 决定总工期
40-20-40	前 40% + 编码 20% + 后 40%	工作量分配规则
人员-时间权衡	$E \propto 1/t^4$	工作量与时间的 4 次方成反比
Brooks 定律	向延期项目加人 = 更晚	通信以 $n(n-1)/2$ 增长
Pareto 80-20	80% 的风险来自 20% 的已知风险	风险管理的优先级依据